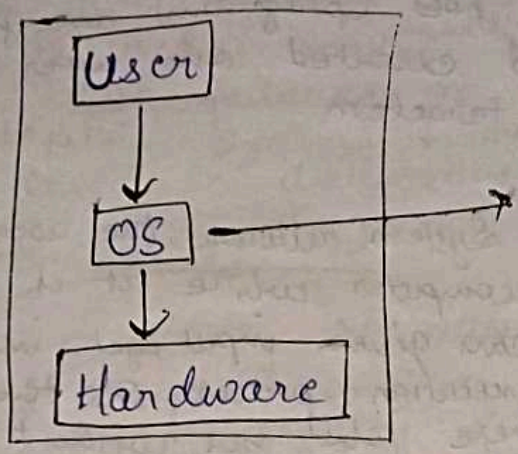
OPERATING SYSTEM

↳ OS is like a manager without it your programs can't run and you can't interact with the computer easily



An OS is the main software that acts as a bridge between computer hardware and user. It manages how hardware (CPU, memory, storage) and software (applications) work together.

functions of OS -

(i) Process management - Runs multiple programs.
(ii) Memory management - keeps track of which program is using which memory
(iii) File System management - Stores data in files and folder
(iv) Device management - Control I/O devices
(v) Security & Access control - protects data from unauthorized access
(vi) User interface - Provides a way to interact.

Ex windows, Linux, macOS, iOS, Android.

There are 7 main types of OS -

(i) Batch OS
(ii) Interactive OS
(iii) Multiprogramming OS
(iv) Multitasking OS
(v) Multiprocessing OS
(vi) Real-time OS.
(vii) Time-sharing OS.

(i) Batch OS

A batch OS is one of the oldest types of OS. In this similar types jobs (programs) are grouped together into batches and executed one after another without user interaction.

(ii) Interactive OS

An Interactive Operative System allows the user to directly interact with the computer while it is executing tasks. User can give input, get immediate output and control the execution process. It's the opposite of Batch OS, where jobs run without user involvement; Windows, Linux, macOS. Allows multitasking.

(iii) Multiprogramming OS.

A multiprogramming OS allows multiple program to be loaded into memory and executed by the CPU one after another. When one program is waiting (like for input/output), the CPU switches to another program, so the processor never sits idle

(iv) Multitasking OS

A multitasking OS allows a single user to run multiple tasks (programs) at the same time on one computer. The CPU switches between tasks so quickly that it looks like they are running simultaneously

(v) Multiprocessing OS

A multiprocessing OS support a computer system with two or more CPUs (processors). These CPUs work together, sharing the same memory and tasks, so multiple processes can actually run at the same time.

(ii) Real-time OS -

A Real Time OS is designed to process data and give results within a strict time limit. It is used in systems where even a small delay can cause failure or danger. In an airplane's autopilot system, if the OS delays response, it could be disastrous.

(iii) Time-Sharing OS

A time sharing OS allows multiple users to use a single computer at the same time. The CPU time is divided into small time slices, and each user or task gets a turn. It creates the illusion that all users are working simultaneously, even though the CPU is rapidly switching between them.

What is booting process.

The booting process is the sequence of operations a computer performs when it is powered on or restarted to load the operating system into memory and make the system ready for use.

Intel process configuration -

(i) i3

2-4 cores, lower clock speed.
MS office, watching videos, web browsing
budget friendly laptops / desktops -

(ii) i5

4-6 cores, supports multitasking, light gaming, programs

(iii) i7

6-8 cores

(iv) i9

8-24 cores

Types of Software

(i) System Software → Utility Software
(ii) Application Software
(iii) Programming Software

(i) Controls and manages computer hardware
Helps other software run smoothly
Ex - OS → windows, linux, macOS,
Utility programs → antivirus, file manager, disk cleanup

(ii) Designed to perform specific tasks for users
Ex: MS word, Excel, Powerpoint, Chrome, firefox, Games

(iii) Provides tools for developers to write, test, compile
Ex, Turbo C++, Java JVM.

#Difference b/w multitasking and multiprogramming

feature	Multiprogramming	Multitasking
Definition	Technique where multiple programs are loaded into memory, and CPU executes one while other wait	Technique where a single user can run multiple tasks simultaneously by rapidly switching CPU between them
Main Goal	Maximize CPU utilization (efficiency)	Improve user experience and provide interactive computing
User Interaction	Minimal, often batch processing	High, designed for interactive system
Execution	CPU executes one programs at a time, switches where I/O occurs.	CPU switches tasks very quickly (time-sharing)
Scheduling	Based on job selection for CPU usage	Based on time-slicing (each task gets a small CPU time)

Kernel (Heart of your OS).

A kernel is the core part of an operating system. It acts as a bridge between hardware and software. It manages system resources like CPU, memory and I/O devices. All major OS operations (process management, memory management, device handling) are controlled by kernel.

functions –

(i) Process management – Create, schedule and terminate process

(ii) Memory management – Allocate and manage memory space for progeams.

(iii) Devices Management – Control input/output devices w/a device drivers

(iv) File System management – Handle file storage, access and retrieval

(v) Security and Access Control – Protect data and System resource

Types of kernel

(i) Monolithic kernel

All OS services run in kernel space, like process managment, file System management, memory managent. Basically everything is inside one large block of code that runs in priviledged mode (kernel mode)

Key features

Singe large block, high performance, direct access to hardware, less context switching, difficult maintenance

Advantage

- fast execution → no extra context switches
- efficient communication → services can call other directly
- Good for performance-critical performance → common in servers and mainframes.

Disadvantages

- Hard to maintain - large and complex.
- one bug (eg in a driver) can crash the whole system
- Poor security - all services share kernel space.

(ii) Microkernel

A kernel that keeps only the most basic functions (like CPU Scheduling, communication, memory management) inside the kernel, and moves other services (drivers, file system, etc.) to user space.

Advantages

- More secure,
- More stable
- Easier to maintain, extend
- ~~Faster than pure microkernel~~
- A driver crash won't crash the whole OS.

Disadvantages

- Slower than monolithic kernel (because of frequent communication b/w user space and kernel space)
- More complex design

(iii) Hybrid Kernel

A kernel that is a mix of monolithic and microkernel. It keeps some services (drivers, file systems) in kernel space for speed, but also uses microkernel ideas for modularity

Advantages

- Faster than microkernel
- More modular & stable than monolithic
- Balanced performance & security.

Disadvantages

- Larger and less secure than microkernel
- More complex than monolithic
- Can become "monolithic in disguised".

Windows, macOS uses hybrid kernel.

#THREAD

A thread is the smallest unit of CPU execution within a process. It is a lightweight process that runs inside a process. Each process must have at least one thread (called the main thread). Multiple threads inside the same process can run ~~accordingly~~ concurrently and share resources like memory, files and data.

Example

Imagine a web browser (the process):

- One thread loads the web pages.
- Another thread plays music.
- Another thread handles typings.

⇒ All these threads work together but share the same memory space of the browser process.

Types of Threads

(i) User-level threads (ULT)

(ii) Kernel-level threads (KLT)

(i) ULT

Managed by the user program (not the OS)
Thread library handles creation, scheduling and synchronization. OS doesn't know about threads, it only sees the main process.

Advantages

- Very fast (no system call needed)
- Portable (same library can work on diff OS)

Disadvantages

- If one thread blocks (eg. I/O) the whole process blocks
- Cannot take advantage of multiple CPUs directly

(ii) KLT

Managed directly by the OS kernel. Kernel schedules threads individually (not just the process)
Each thread is known to the OS.

Advantages

- If one thread blocks, others can continue
- Can run on multiple CPUs in parallel.

Disadvantages

- Slower (because system calls are needed)
- More overhead for the OS.

PROCESS

Process = Heavy, independent, isolated
Thread = light, fast, shares memory inside a process.

A process is a program in execution, consisting of the program code, current activity (program counter, register) memory and resource allocated by the operating system.

Basically, A process = a running instance of a program

Aspect	Process	Thread
Definition	A program in execution, managed by the OS	A lightweight unit of execution inside a process.
Memory Space	Each process has its own separate memory space (address space)	Threads of the same process share memory and resource
Overhead	High overhead (context switching is slower and costlier)	Low over (context switching is faster)
Communication	Needs inter-process communication (IPC) like Pipes, sockets, messages.	Threads can communicate easily through shared memory (faster)
Isolation	Processes are independent crash of one does not affect others.	Threads are dependent → crash of one thread may crash the whole process
Creation time	Slower to create (needs separate resources like memory, PCB)	faster to create (shares process resource)
Examples	Running MS Word and Chrome are two different processes	Inside Chrome, different tabs or tasks (video, input, rendering) are threads.

Interrupt

An interrupt is a signal sent to the CPU by hardware or software that temporarily stops the execution of the current instruction so the CPU can handles an important event. After the event is handles, the CPU resumes the previous task.

Keypoints

- Generated by hardware devices (I/O, disk, keyboard)
- Can also be generated by software.
- Used for asynchronous events (events happening outside the CPU's current instruction flow).

Example

- Pressing a key on the keyboard → keyboard interrupt
- Timer interrupt → used for scheduling processes
- Disk I/O completion interrupt → CPU is notified when data is ready

Trap

A trap is synchronous interrupt caused by a program (software) during execution. It occurs intentionally (System call) or due to an error/exception in the program.

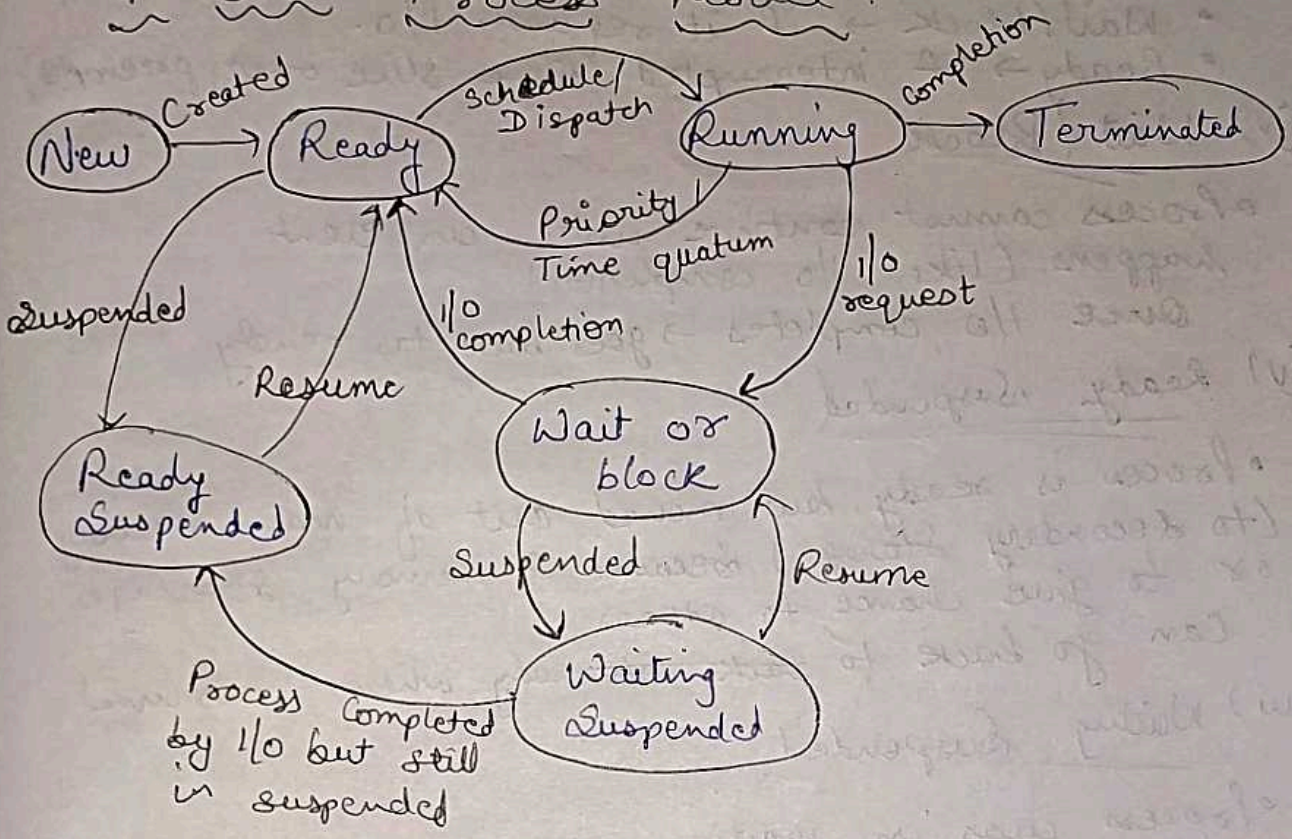
Keypoint

- Generated by software instructions.
- Used for system calls (to request OS services)
- Can also signal exceptions (illegal instructions, divide by zero, memory errors).

Examples

- Divides by zero error → causes a trap
- Invalid memory access → trap

7 Star Process Model



A process (program in execution) can be in different states depending on what it is doing (running, waiting, etc) - The 7 states are:

(i) New

- Process is just created
- OS has accepted it, but it is not yet ready for execution

Goes to Ready state once admitted

(ii) Ready

- Process is loaded into main memory
- Waiting in Ready queue for CPU time

Scheduler pick it → goes to Running

(iii) Running

- Process is actually being executed by CPU
  Can change to:
- Terminated → if completed
- Wait/block → if it request I/O.
- Ready → if interrupted (time slice over, preempted)

(iv) Wait / Block

- Process cannot continue until an event happens (like I/O completion).
  Once I/O completes → goes back to ready

(v) Ready Suspended

- Process is ready, but moved out of main memory (to secondary storage) because of memory shortage or to give chance to others.
  Can go back to ~~back wbs~~ ready when resumed

(vi) Waiting Suspended

- Process was in waiting state, but suspended (kept in secondary memory)
  Stays here until event completes and it is resumed

(vii) Terminated

- Process has finished execution and is removed from the system.

Easy way to remember.

- New → Ready → Running → Terminated (normal flow)
- If I/O request → Waiting
- If Suspended → goes to Suspended Ready/ Suspended Waiting